

MANUAL TÉCNICO

Manual de despliegue en la nube

Contenerización, ejecución local y publicación administrada del sistema de clasificación

Campo	Detalle
Proyecto	Clasificación de cáncer de mama - Breast Cancer Wisconsin Diagnostic
Repositorio	https://github.com/HamDan314/cancermm
Responsable	Leonel Cardoso Gómez
Fecha	2 de agosto de 2026
Carácter	Proyecto académico; no constituye diagnóstico médico

1. Objetivo y alcance

Este manual describe cómo construir, ejecutar, validar y desplegar una solución académica de clasificación de tumores. La arquitectura separa el frontend estático, la API de inferencia y el artefacto del modelo para mejorar portabilidad, trazabilidad y mantenimiento.

2. Arquitectura de la solución

Componente	Tecnología	Responsabilidad
Frontend	HTML, CSS, JavaScript y Nginx	Captura 30 características, consume /api/predict y muestra el resultado.
Backend	Python 3.11 y FastAPI	Carga el modelo, valida solicitudes y expone endpoints REST.
Modelo	scikit-learn 1.8.0 y joblib	Clasifica el registro como maligno o benigno y devuelve probabilidades.
Orquestación	Docker Compose	Construye y conecta frontend y backend en una red reproducible.
Integración continua	GitHub Actions	Ejecuta Pytest y construye las dos imágenes Docker.

Flujo operativo: usuario → Nginx (puerto 8080) → proxy /api → FastAPI (puerto 8000) → modelo joblib.

3. Requerimientos técnicos

Recurso	Requerimiento
Sistema operativo	Windows 10/11, Linux o macOS con soporte para contenedores.

Recurso	Requerimiento
Docker	Docker Desktop 4.x o Docker Engine 24 o superior.
Compose	Docker Compose v2.
Git	Git 2.40 o superior.
Memoria	2 GB de RAM disponibles como mínimo.
Puertos	8000 para la API y 8080 para el frontend.
Cuenta de nube	Cuenta activa en Google Cloud, Render, Railway o plataforma equivalente.

4. Ejecución local paso a paso

1. Clonar el repositorio: `git clone https://github.com/HamDan314/cancermm.git`
2. Entrar al proyecto: `cd cancermm`
3. Construir imágenes limpias: `docker compose build --no-cache`
4. Iniciar servicios en segundo plano: `docker compose up -d`
5. Comprobar estado: `docker compose ps`
6. Abrir `http://localhost:8080` y realizar una predicción con el caso de ejemplo.
7. Consultar Swagger en `http://localhost:8000/docs` y salud en `http://localhost:8000/health`.
8. Detener los servicios al finalizar: `docker compose down`

5. Construcción y publicación de imágenes

Construcción independiente:

- `docker build -t hamdan314/cancer-api:1.0.0 ./backend`
- `docker build -t hamdan314/cancer-frontend:1.0.0 ./frontend`

Publicación en un registro compatible con OCI:

- `docker login`
- `docker push hamdan314/cancer-api:1.0.0`
- `docker push hamdan314/cancer-frontend:1.0.0`

6. Estrategia de despliegue

6.1 Contenedores administrados

La opción recomendada es Google Cloud Run, Azure Container Apps o AWS App Runner. Estas plataformas ejecutan imágenes OCI, proporcionan HTTPS, escalan bajo demanda y evitan administrar servidores.

Criterio	Contenedor administrado	PaaS desde GitHub
Control	Alto: se despliega una imagen versionada.	Medio: la plataforma construye desde el repositorio.
Operación	Escalado y HTTPS administrados.	Flujo simplificado y despliegue automático.
Portabilidad	Alta entre proveedores compatibles con OCI.	Depende más de la configuración del proveedor.
Uso recomendado	Producción y trazabilidad de versiones.	Demostraciones, prototipos y entregas académicas.

6.2 Backend en Google Cloud Run

9. Autenticarse: `gcloud auth login`
10. Seleccionar el proyecto: `gcloud config set project ID_PROYECTO`
11. Desplegar la imagen del backend con puerto 8000, 1 GiB de memoria y acceso HTTPS público.
12. Registrar la URL generada y comprobar `/health` y `/model-info`.

6.3 Alternativa PaaS

Render o Railway pueden desplegar directamente desde GitHub. Para el backend se configura la carpeta raíz backend, entorno Docker, health check `/health` y variable `PORT`. Para el frontend se configura la carpeta raíz frontend y el puerto interno 80.

7. Configuración del frontend

En Docker Compose, el navegador llama a `/api` y Nginx reenvía al servicio backend:8000. Este patrón evita una URL fija y reduce problemas de CORS. Si frontend y backend se publican como servicios separados, se sustituye `API_URL` en `frontend/app.js` por la URL HTTPS pública del backend antes de reconstruir la imagen.

8. Seguridad y operación

- Restringir CORS a la URL real del frontend.
- Mantener HTTPS administrado por la plataforma.
- Usar etiquetas semánticas de imagen y evitar depender únicamente de `latest`.
- Ejecutar el backend con usuario sin privilegios, como define el `Dockerfile`.
- No registrar datos clínicos o identificadores personales en los logs.
- Configurar límites de CPU, memoria y alertas para errores HTTP 5xx.
- Añadir autenticación antes de procesar información médica real.

9. Validación posterior al despliegue

Prueba	Criterio de aceptación
GET /health	HTTP 200, status healthy y model_loaded true.
GET /model-info	Versión, 30 variables y métricas del modelo.
POST /predict	HTTP 200, clase, probabilidades, latencia y versión.
Frontend	Resultado mostrado sin errores de red o CORS.
GitHub Actions	Pruebas y construcción de ambas imágenes en estado successful.

10. Conclusión

La estrategia basada en contenedores mantiene el mismo entorno entre desarrollo, integración continua y nube. Docker Compose facilita la comprobación local; un servicio administrado reduce la carga operativa; y GitHub Actions aporta evidencia reproducible de pruebas y construcción de imágenes.